



L3 informatique

Course Notes

Software Engineering

Lecture I Introduction

Presented by
Mr KHELIFA N.

Introduction

- **Software Engineering was defined during:**
 - An international conference held from October 7 to 11, 1968, in Garmisch-Partenkirchen, Germany, under the sponsorship of NATO.
- **The conference was convened following the observation of:**
 - The rapid growth of hardware performance (due to the use of semiconductors),
 - The emergence of the so-called “*software crisis*.”
- **The software crisis was characterized by:**
 - Failure to meet delivery deadlines,
 - Budget overruns,
 - Deliverables that did not meet user or client requirements,
 - Software systems that were difficult to use, maintain, and evolve.
- **The proposed solution:**
The industrialization of software production, involving:
 - The organization of production processes (life cycle models, methods, notations, tools),
 - The structuring of development teams,
 - The establishment of rigorous quality assurance plans.
- **The software crisis has not yet been fully resolved to this day.**

software engineering

- **Definition of Engineering:**

The application of a systematic, disciplined, and quantifiable approach to the design, construction, and operation of structures, machines, products, systems, or processes.

- **Definition of Software (John Tukey, 1958):**

- **Definition 1:** Programs, procedures, and possibly associated documentation and data related to the operation of a computer system.
- **Definition 2:** All or part of the programs, procedures, rules, and associated documentation pertaining to an information processing system.

Software Engineering

- **Definition of Software Engineering**

- **Definition 1 [IEEE]:**

The application of a systematic, disciplined, and quantifiable approach to the development, operation, and maintenance of software; that is, the application of engineering to software.

- **Definition 2:**

The systematic application of scientific and technological knowledge, methods, and practical experience to the design, implementation, testing, and documentation of software.

- **Objectives (FCDQ):**

- The developed system must ensure the required functionalities.
 - Development costs must remain within the initially defined limits.
 - Deadlines must be met as initially planned.
 - The quality of the software must comply with the initial service agreement.

- **Key dimensions:** deadlines, costs, and quality.

Principles of Software Engineering

- **Principles of Software Engineering**
 - Principles are rules to be followed. They embody the collective wisdom of many practitioners who have learned through experience. They are present across all methods employed in Software Engineering.
- **The principles of software engineering are characterized by the following aspects:**
 - They are fundamental and invariant in their application.
 - They are less specific than methods and techniques.
 - They are more enduring than methods and techniques.
 - They should be derived from practical experience.

Principles of Software Engineering

- **Principle I: Rigor**

- Software activities must be carried out rigorously through:
 - Correct use of appropriate techniques,
 - Delivery of the expected artifacts (documents, models, code),
 - Validation of all deliverables,
 - Professional conduct within the team,
 - Adherence to suitable processes,
 - Task automation to reduce errors caused by human actions, by selecting an appropriate level of specification formality and tools,
 - Utilization of tools such as proof assistants, code generators, and test generators.

- **Levels of Rigor**

- **Formal Specification:** Represents the highest level of certification; validation relies on mathematically proven demonstrations.
- **Semi-formal Specification:** Introduces standardized languages to describe the software and its specifications.
- **Textual Specification:** Often incomplete, ambiguous, redundant, and unstructured, making verification and validation of descriptions very difficult and error-prone.

Principles of Software Engineering

- **Principle 2: Modularity**

- A system is modular if it is composed of simpler subsystems or modules. Modularity allows one to consider separately the content of each module and the relationships between modules
- It facilitates reuse, adaptability, and teamwork.
- A good modular decomposition is characterized by **high cohesion** and **low coupling**.

- **Principle 3: Separation of Concerns**

- This principle refers to decomposing a problem into independent sub-problems in order to:
 - Separate issues so that only one is addressed at a time,
 - Simplify problems and manage complexity progressively.
- It is closely related to the principle of modularity, meaning that:
 - A single concern is encapsulated within a given module.

- **Principle 4: Incremental Construction**

- An incremental process achieves its objective step by step, with each result being built by extending the previous one, progressively approaching the final system.

- **Principle 5: Abstraction**

- Abstraction is a mechanism that allows the representation of a context by emphasizing the relevant elements while omitting those that are not.

Principles of Software Engineering

- **Principle 6: Generalization and Genericity**

- It is sometimes advantageous to replace the solution of a specific problem with the solution of a more general one, such as grouping a set of similar functionalities into a single parameterized functionality.

- **Principle 7: Structuring**

- This refers to the way software is decomposed, using either a bottom-up or a top-down approach.

- **Principle 8: Documentation**

- Corresponds to the management of documents, including their identification, acquisition, production, storage, and distribution.

- **Principle 9: Verification**

- The process of determining compliance with the specifications established based on the requirements identified in the previous phase of the life cycle.

- **Principle 10: Anticipation of Evolution**

- Ensuring that necessary modifications can be made as locally as possible.

Software Quality – Quality Criteria

- **Definition 1:**

- The capability of a software product to satisfy stated and implied needs under specified conditions.

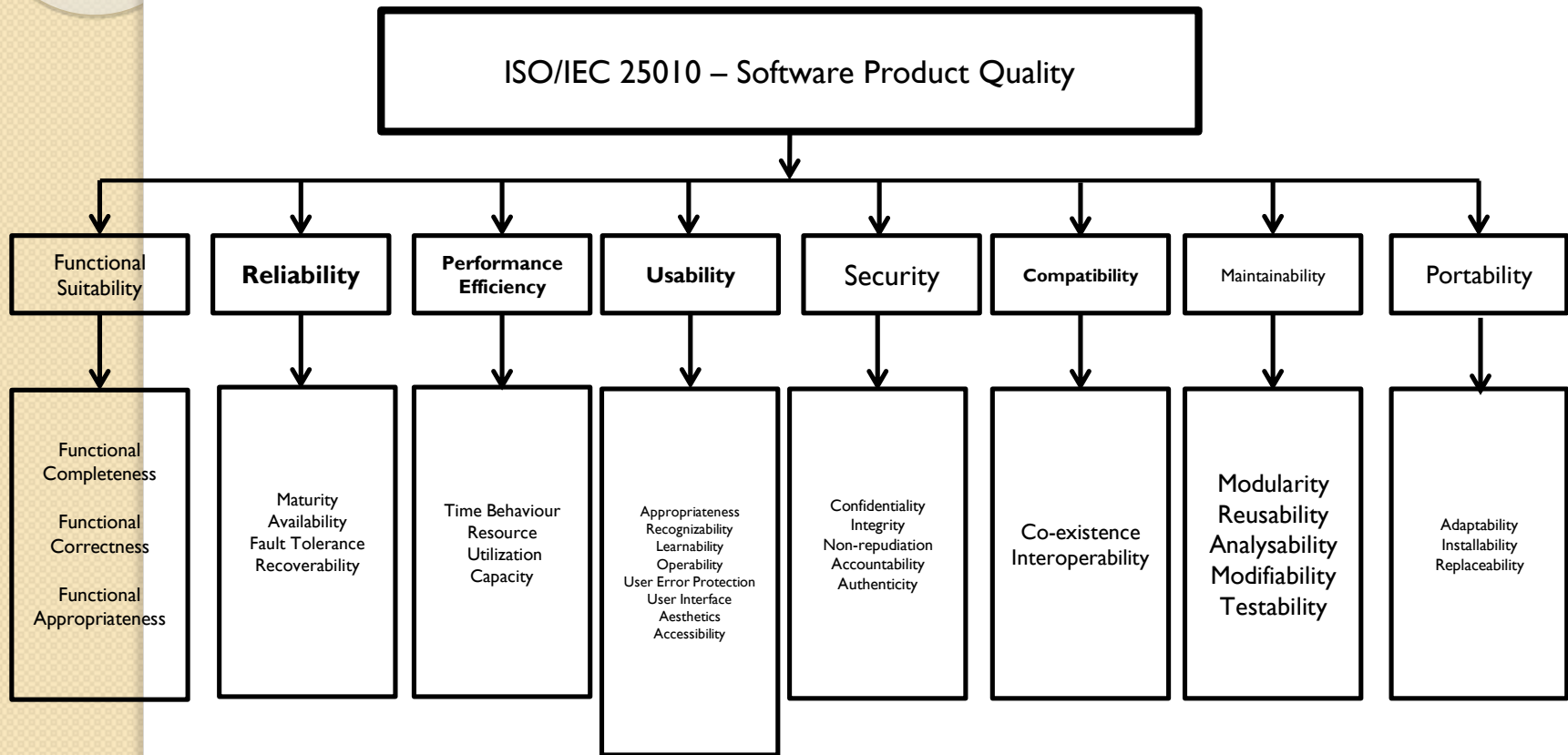
- **Definition 2:**

- The degree to which stated and implied needs are satisfied under specified conditions.

- **Software Quality Assurance (SQA):**

- A set of planned and systematic activities encompassing all necessary actions to provide sufficient confidence that a developed or modified software product conforms to established requirements and expectations.

Software Quality – Quality Criteria



Software Quality – Quality Criteria

- **Functional Suitability** – The degree to which the software provides functions that meet stated and implied needs.
- **Performance Efficiency** – The performance relative to the amount of resources used under stated conditions.
- **Compatibility** – The ability of the software to coexist and interact with other products, systems, or environments.
- **Usability** – The extent to which the product can be used effectively, efficiently, and satisfactorily by specified users.
- **Reliability** – The ability of the software to maintain its level of performance under stated conditions for a given period.
- **Security** – The degree of protection of information and data, ensuring confidentiality, integrity, and availability.
- **Maintainability** – The ease with which the software can be modified, corrected, or adapted.
- **Portability** – The ability of the software to be transferred from one environment to another.

Software Life Cycle (SLC)

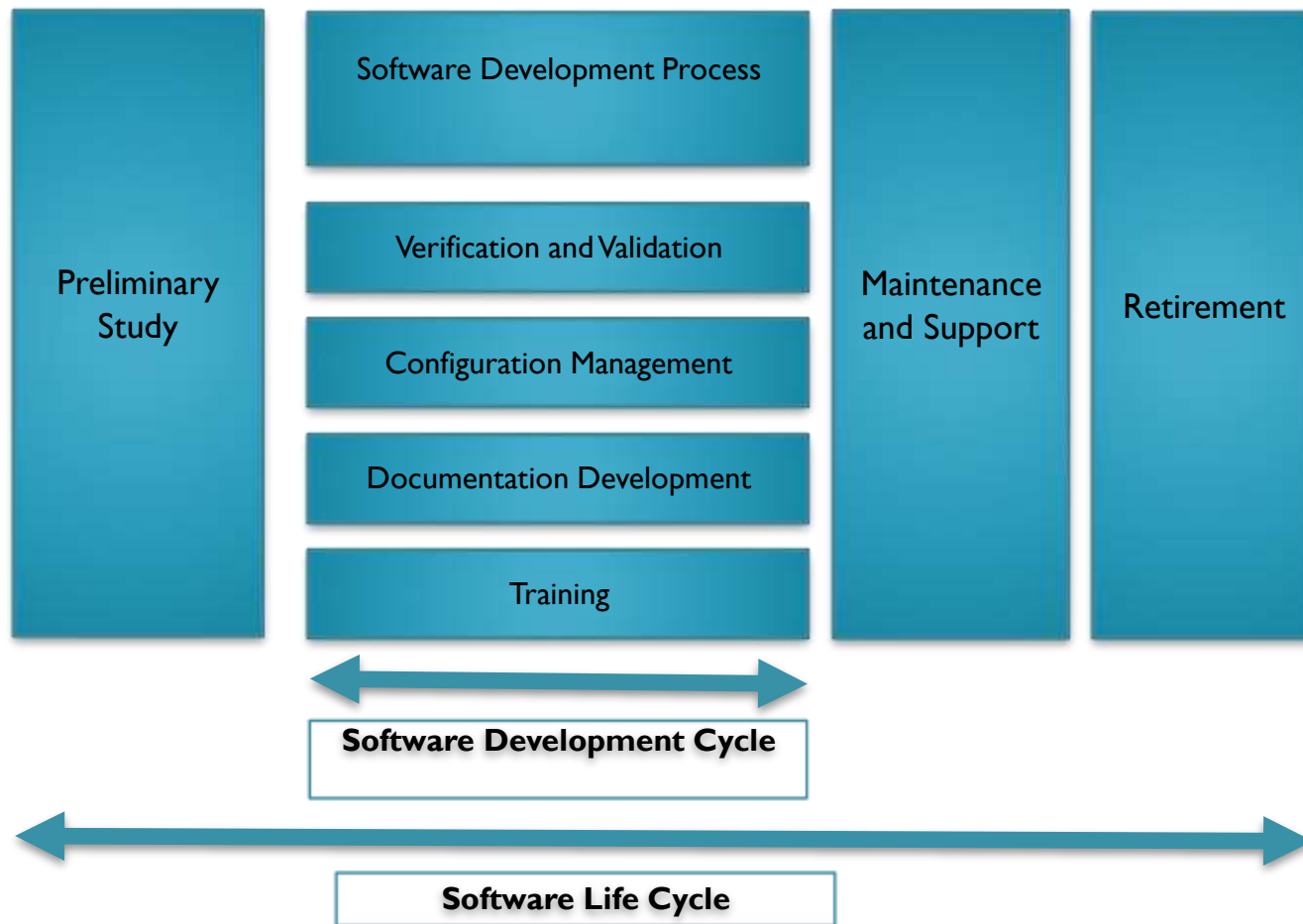
- **Software Life Cycle (SLC):**

It is the time span covering the entire software process, from its beginning to its end.

It encompasses all phases and stages, from the proposal or decision to develop a software system to its decommissioning.

- This structuring makes it possible to define intermediate milestones that allow for:
 - The validation of software development (ensuring that the software conforms to the stated requirements), and
 - The verification of the development process (ensuring that the methods applied are appropriate).
- The software life cycle enables early error detection, thereby ensuring control over software quality, development timelines, and associated costs.

Software Life Cycle



Fundamental Activities

Specify

What must the software do?

How can we ensure it does it?

How can we ensure we are developing the right software?

Validate

Does the software do what it is supposed to do?

Are we developing the right software?

Code

How is this organization translated into source code?

Design

How should the software be organized so that it does what it is supposed to do?

What technical choices must be made so that the software does what it is supposed to do?

How can we ensure that the software is organized and built in a way that allows it to do what it is supposed to do?

Integrate

Is the software organized and built in such a way that it does what it is supposed to do?

Unit Test

Is the source code correctly written?

Software Life Cycle Model (SLCM)

- **Software Life Cycle Models (SLCM):**

- They describe, at a highly abstract and idealized level, the different ways of organizing software production.
- An SLCM is characterized by:
 - The stages and their sequencing,
 - The criteria for transitioning from one stage to another,
 - The termination criteria for a stage (e.g., document reviews),
 - The criteria for selecting the next stage,
 - The criteria for initiating a stage.

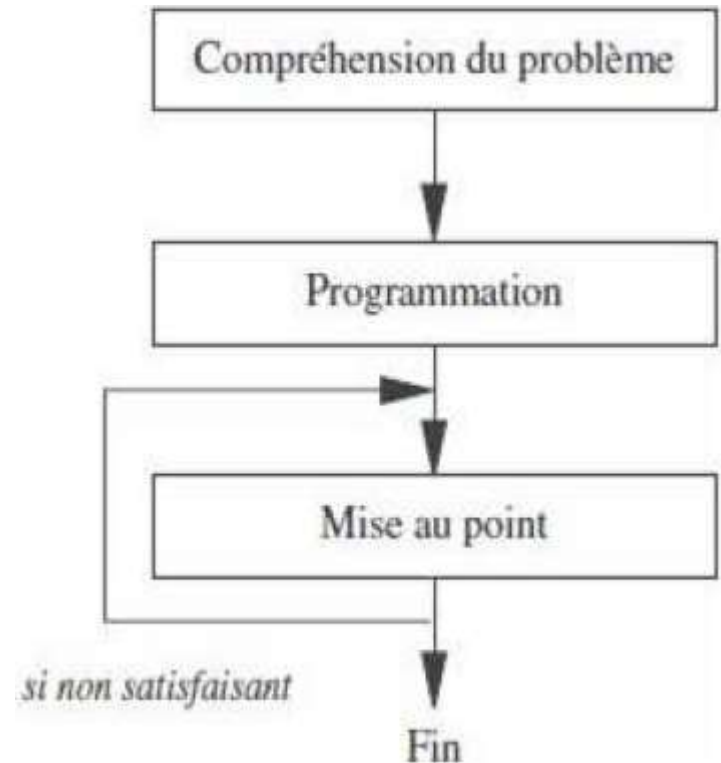


Software Life Cycle Model

- **Code-and-Fix Model**
- **Waterfall Model**
- **V-Model**
- **Prototyping Model**
- **Incremental Model**
- **Spiral Model**

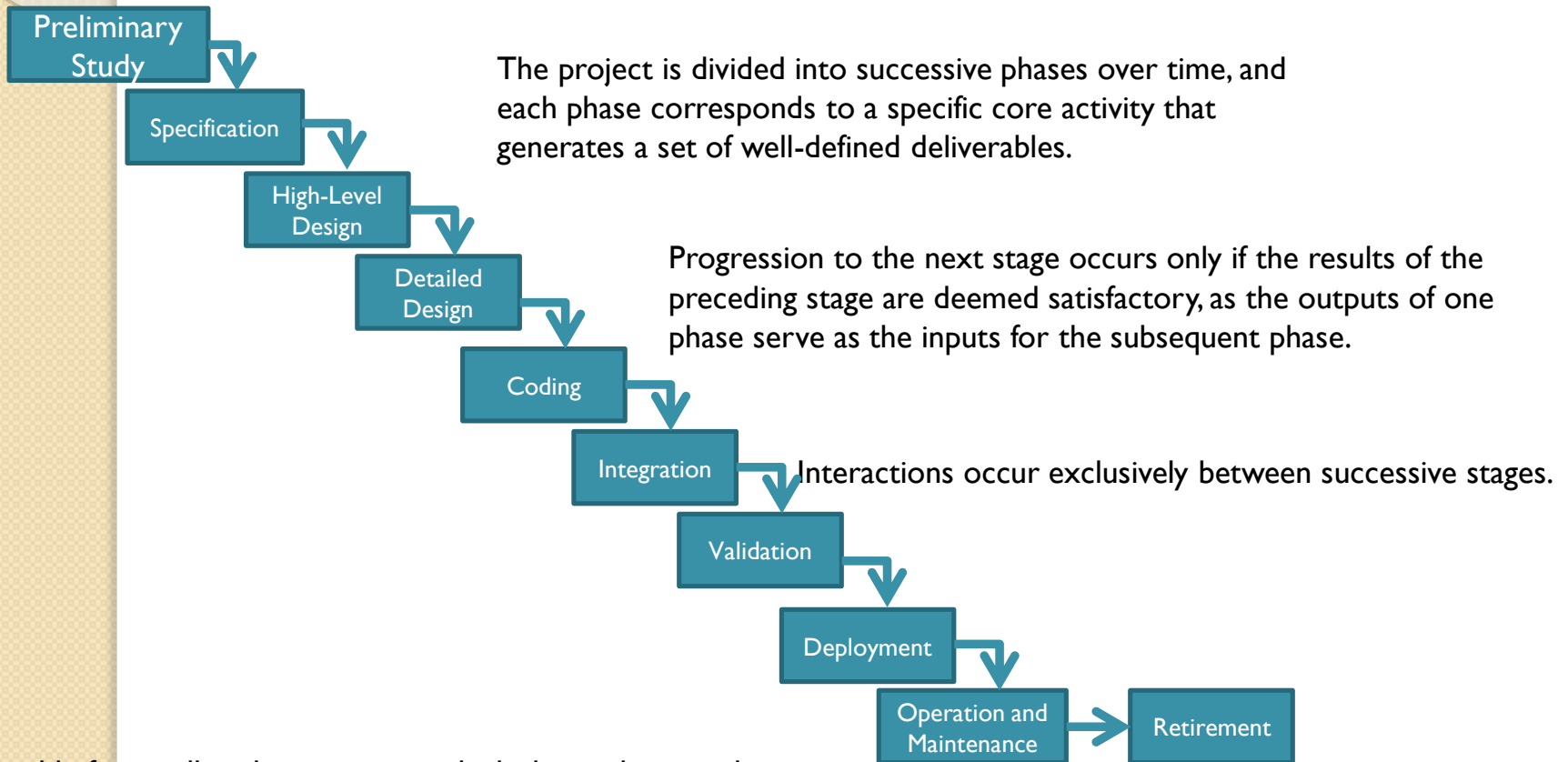
Code-and-Fix Model

- An initial Problem Understanding phase
- A Programming phase
- A Debugging phase, sometimes in collaboration with the future system's user, repeated until the intended result is achieved



Waterfall Model

- This life cycle is linear, with the different software development stages carried out sequentially.



Suitable for small-scale projects in which the application domain is well understood.

Waterfall Model

- **Preliminary Study (Feasibility)**
- **Exploratory Phase**
 - Should the software be developed?
 - Define the general conditions.
 - **Output:** Interview reports, decisions (develop, not develop, purchase, etc.), approximate budget.
- **Design Phase**
 - Project specifications and project plan.
 - **Output:** Specification document, overall project plan, detailed budget, etc.
- **Specification (Requirements)**
- A process that establishes the list of what is expected from the system, as well as the constraints on its execution and development.
- **Requirements Engineering Process:**
 - **Feasibility Study:** Is it technically and financially feasible to build the system?
 - **Requirements Analysis:** What do the system stakeholders expect from the system?
 - **Requirements Specification:** Defining the requirements in detail.
 - **Requirements Validation:** Verifying the validity of the requirements.
 - **Output:** Requirements specification document (functions and performance), first version of the user manual, detailed project plan, validation plan.

Waterfall Model

- **High-Level Design (Product Design)**

- System Architecture
- Main data structures
- Decomposition of the system into modules

□ **Output:** Definition of the main data structures, decomposition of the system into modules (architecture), description of each module's role.

- **Detailed Design**

- Refinement of the previous elements into a form that allows direct program implementation.

□ **Output:** Detailed description of data structures and modules.

- **Coding**

- Writing the program source code.

□ **Output:** Program code; each module is individually tested.

Waterfall Model

- **Integration**

- Assembling the different modules.
- Construction of the overall architecture.
- System-wide validation/acceptance testing.
- ☐ **Output:** Acceptance report, inspection and validation reports.

- **System Validation / Acceptance Testing**

- Verifying that the correct system has been built.
- ☐ **Output:** User satisfaction (or dissatisfaction), product acceptance (or rejection), accepted software, test reports.

- **Deployment (Release/Distribution)**

- Preparation and distribution of the different software versions.
- ☐ **Output:** Program versions and adapted documentation.

- **Operation (Exploitation)**

- Deployment of the system in its operational environment.
- ☐ **Output:** Running software, incident and correction reports.

- **Maintenance**

- Process aimed at keeping the software operational or improving it.
- Maintenance can be:
 - **Corrective** (fixing latent errors)
 - **Adaptive** (adjusting to user requirements)
 - **Evolutionary** (adapting to changes in the environment)
 - **Perfective** (optimization and improvement)
- ☐ **Output:** Maintenance report.

Waterfall Model

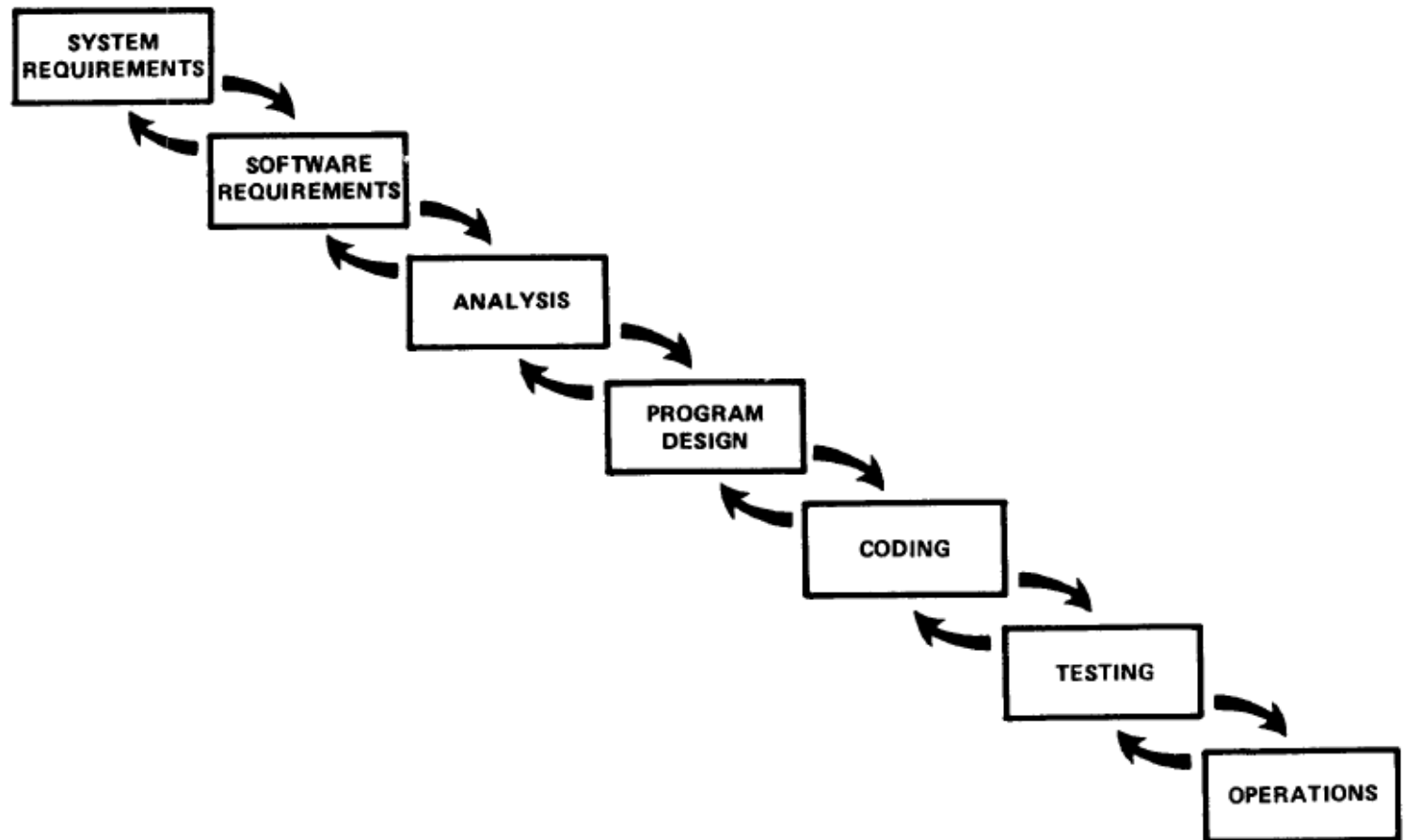
- **Advantages**

- Easy to use and to understand.
- Simple structure for an inexperienced team.
- Works well when quality is far more important than schedule and cost.

- **Disadvantages**

- Sensitive to new requirements: the entire process must be redone.
- A phase cannot start until the previous one is completed.
- The product is not visible until the very end.
- Risks are postponed until late in the process.

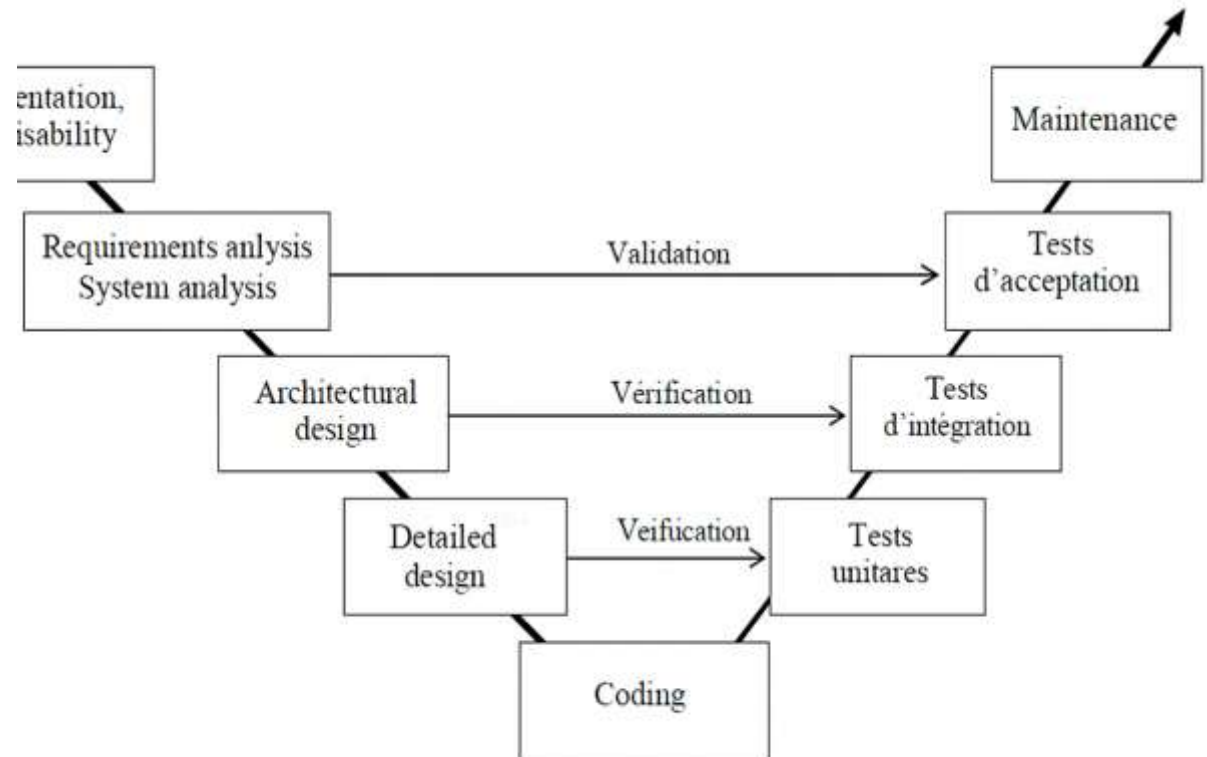
Waterfall Model (with feedback)



V-Model

- **The life cycle is linear.**
There are two branches:
- The first branch corresponds to a classical waterfall model.
- The second branch corresponds to effective testing performed on implemented components.

- Not only shows the sequence of successive phases, but also the logical relationships between more distant phases.



V-Model

- **Advantages**

- Emphasizes testing and validation, thereby increasing quality.
- Each deliverable must be testable.
- Easy to use and to plan.

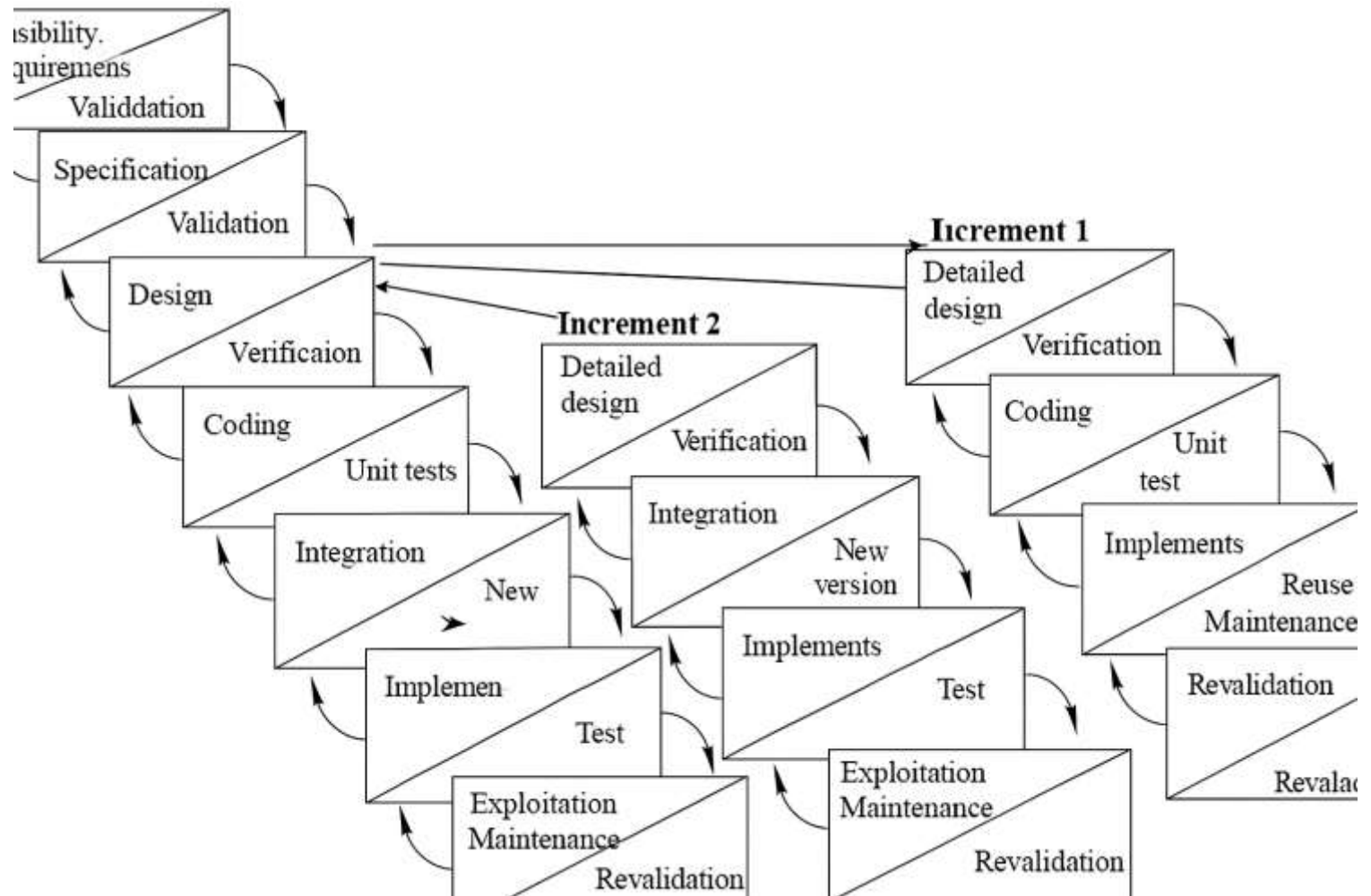
- **Disadvantages**

- Does not handle parallel activities.
- Does not handle changes in specifications.
- Does not include risk analysis activities.

Incremental Model

- Starting from a given phase (generally the architectural design), the process is iterated several times, and each iteration results in the production of increments.
- **An increment**
A tested and deliverable version of a software product (each time getting closer to the final product) that provides new or modified functionalities.

Incremental Model



Incremental Model

- **Advantages**

- High-risk functionalities are developed first.
- Each increment delivers a functional product.
- The customer is involved at the end of each increment.
- Follows the “divide and conquer” approach.
- The customer interacts with the product at a very early stage.

- **Disadvantages**

- Requires careful planning and sound design.
- Requires a clear vision of the final product in order to properly divide it into increments.
- The total system cost may be high.
- May jeopardize the core system or previous increments.
- May prevent the integration of new increments.

prototyping model

- **A prototype**

An executable model of all or part of a software system, easy to implement and modify, which allows rapid verification of certain functionalities of the software while sacrificing the accuracy of others.

It may be built without the same robustness constraints as the final system.

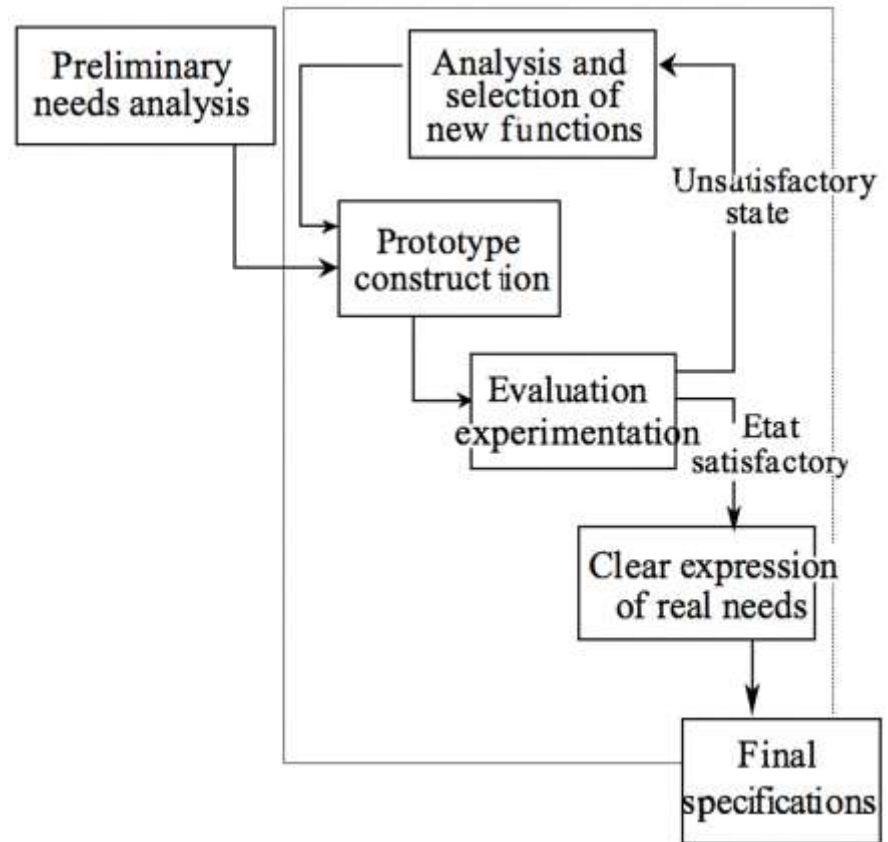
- **The prototyping model**

A software development approach that emphasizes the preparation of working versions used as a basis for evaluating ideas and making decisions, with the aim of preparing the complete version to be delivered to the client.

prototyping model

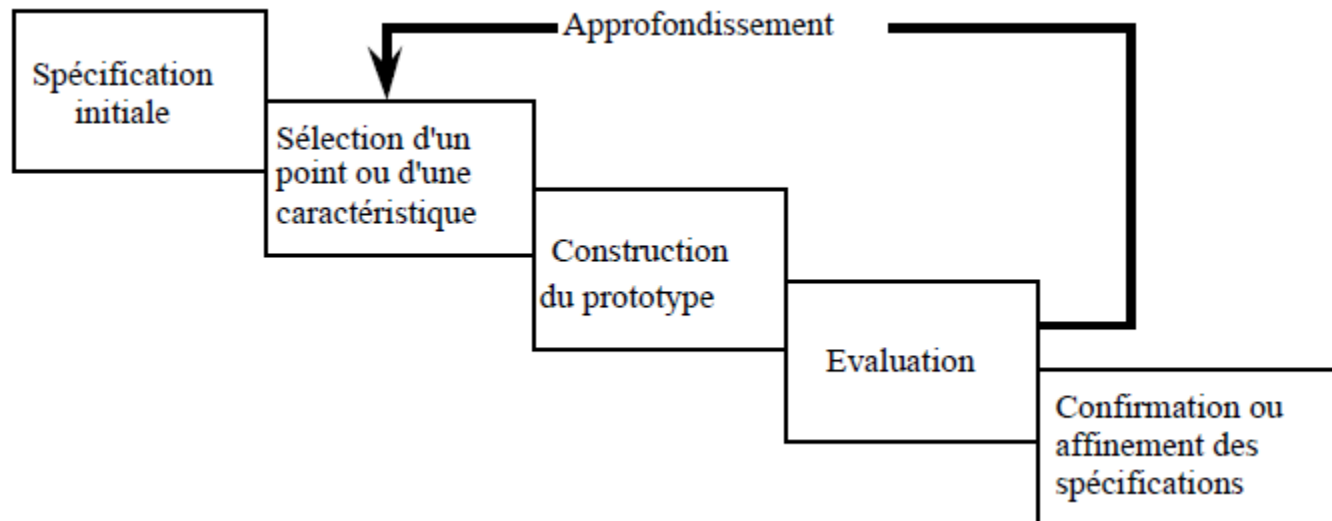
- **Rapid Prototyping (or Mock-up)**

- Used at the early stages of the development cycle:
 - Requirements analysis,
 - Functional specifications.
- It enables the validation of specifications.
- It allows the client and the developer to reach a clear agreement on the nature of the product to be developed, particularly regarding the interface and functionalities.



prototyping model

- **Experimental Prototype**
Used at the design stage to:
 - Ensure the feasibility of critical components.
 - Validate design alternatives.
- **Example:** Prototype of a parser with a reduced grammar.

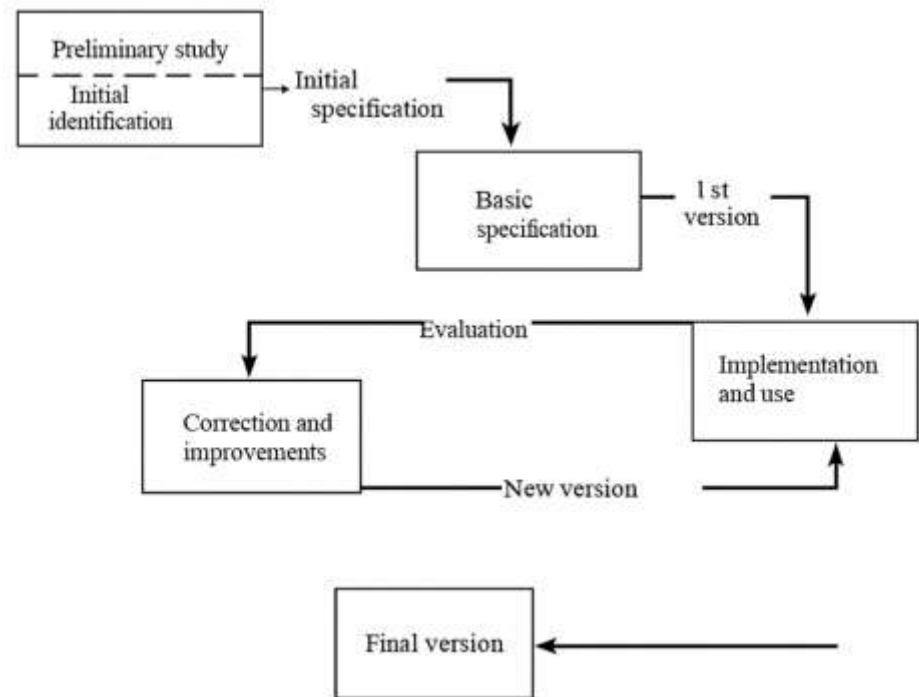


prototyping model

- **Evolutionary Prototype**

- The first version of the prototype represents the embryo of the final product.
- Iterations are carried out until the final product is achieved.

- **Example:**
Development of an expert system



prototyping model

- **The prototyping model** is desirable for projects in which the requirements:
 - Are not clearly defined.
 - Are likely to change over time.
- **Types of prototypes**
 - **Throwaway prototypes:** Help in better understanding customer requirements, evaluating alternative solutions, etc.
 - **Evolutionary / reusable prototypes:** Incrementally refined during subsequent iterations until the final product is obtained.
- **Iterative development:** Produces a complete system at each iteration.
- **Incremental development:** Produces a partial system that is progressively completed at each iteration.

prototyping model

- **Advantages**

- Enables early focus on critical aspects and areas of uncertainty in the development process.
- Simplifies the elaboration of specifications and the Human–Computer Interface (HCI).
- Evolutionary and incremental.
- Can be combined with other models and applied at different phases, with the prototype objectives varying accordingly.
- Well suited for innovative projects.

- **Disadvantages**

- Many issues related to project management (cost, time, monitoring, control, planning, etc.).

Spiral Development Model

- **The Spiral Development Model**

- is a process model generator driven by risk considerations. It has two main distinctive characteristics:

- A cyclic approach that progressively increases the degree of definition and implementation while simultaneously reducing the level of risk.
 - A set of milestones to ensure stakeholder commitment to feasible and mutually satisfactory system solutions.

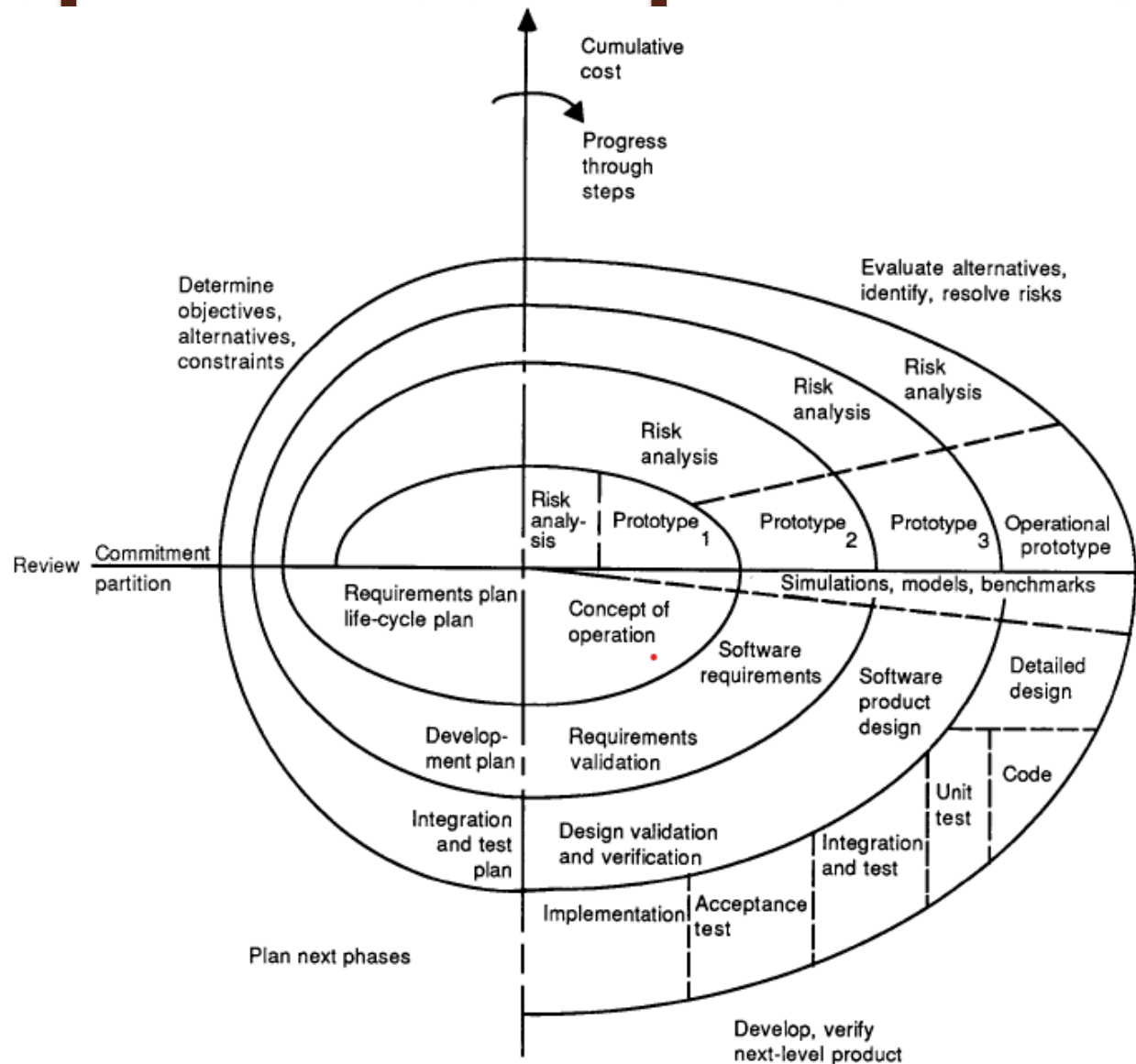
- **Risks**

Are situations or potential events that may prevent a project from achieving its objectives.

- **A risk management plan**

- Lists the risks and prioritizes them according to their level of importance.
 - For each risk, the plan also specifies a mitigation strategy to address it.

Spiral Development Model



Spiral Development Model

- **Determination of cycle objectives, alternatives to achieve them, and constraints:**
based on the results of previous stages or preliminary requirements analysis.
- **Risk analysis and evaluation of alternatives:**
using mock-ups and/or prototypes.
- **Development and verification of the chosen solution:**
a “classical” model (waterfall or V-model) can be used at this stage.
- **Review of results and preparation for the next cycle.**

Risk	Remedy
Personnel failure	Recruitment of high-level staff, mutual training, leadership, adequacy of profile/function
Unrealistic schedule	Detailed estimation, incremental development, reuse, requirements trimming
Financial risk	Cost/benefit analysis, cost-conscious design
Development of inappropriate functions	User reviews, early user manual...
Development of inappropriate interfaces	Prototyping, task analysis
Requirements volatility	Incremental development of the most stable parts first, information hiding
Performance issues	Simulations, modeling, testing and measurement, prototyping
Excessive requirements compared to technology	Technical feasibility analyses, prototyping
Failure of external tasks or components	Subcontractor audits, contracts, reviews, compatibility analysis, testing and measurement...

Spiral Development Model

- **Advantages**

- Incorporates risk analysis and prototyping.
- Critical functions are developed first.
- Provides rapid client feedback.
- Continuous evaluation of the process.
- Each cycle includes the same activities as the waterfall model.

- **Disadvantages**

- Risk assessment can be time-consuming.
- The model is very complex.
- Schedules and budgets are often unrealistic.
- Difficult to anticipate components needed in subsequent cycles.
- Implementation requires significant expertise and should be limited to innovative projects due to the emphasis on risk analysis.